

Comprehensive Repository Analysis

Purab-Gaonkar / pneumonia-detection

Generated 2026-04-30

1. Executive Summary and Project Goals

PneumoScan AI is a full-stack medical imaging application designed to classify chest X-ray images as either Normal or Pneumonia. The project combines three pretrained CNNs into a late-fusion ensemble model (DenseNet121, ResNet50V2, MobileNetV2), adds an Out-of-Distribution (OOD) rejection guard powered by an Isolation Forest, and delivers results through a polished glassmorphism-styled web interface backed by a FastAPI asynchronous server.

Primary Goals

- Provide a turnkey, demo-ready pneumonia screening tool using transfer learning.
- Prevent nonsensical predictions by rejecting non-X-ray uploads via the OOD detector.
- Package the solution as a locally-hosted web app requiring no cloud services.
- Keep the codebase concise and educational with an explicit medical disclaimer.

Scope: The project is intentionally scoped as a research/education prototype; it is not validated for clinical deployment.

2. Repository Structure

```
pneumonia-detection/
??? README.md           # Installation, usage, and disclaimer
??? requirements.txt     # Python package dependencies (7 entries)
??? .gitignore          # Excludes models/, chest_xray/, dataset/, venv/
??? train.py            # End-to-end training script (166 lines)
??? app.py              # FastAPI inference server (103 lines)
??? scratch.py          # OOD batch-validation utility (47 lines)
??? test_upload.py      # HTTP smoke-test script (9 lines)
??? static/
  ??? index.html        # Single-page web UI (265 lines)
  ??? script.js         # Frontend logic (166 lines)
  ??? style.css         # Glassmorphism dark theme (838 lines)
```

Notable Absences (gitignored by design)

- models/ -- trained artefacts (pneumonia_ensemble_model.h5, ood_isolation_forest.pkl)
- chest_xray/ -- dataset directory
- dataset/ -- alternative dataset directory
- venv/ -- virtual environment
- scratch.py and test_upload.py are gitignored, indicating developer-only utilities

There are no notebooks, CI configuration files, Docker files, or test suites.

3. Data Pipeline Assumptions

3.1 Dataset Source

README line 24: "The dataset is not included in this repository. Place your dataset directory relative to the project or adjust the DATASET_DIR variable in train.py." The default DATASET_DIR = "chest_xray" (train.py line 20) and references to train/val subdirectories strongly indicate the Kaggle Chest X-Ray Images (Pneumonia) dataset by Paul Mooney, the canonical open benchmark with that folder name and binary class structure.

3.2 Expected Folder Layout

```
chest_xray/
??? train/
?   ??? NORMAL/
?   ??? PNEUMONIA/
??? val/
    ??? NORMAL/
    ??? PNEUMONIA/
```

Note: No test/ split is consumed. scratch.py globs val/*/*.jpeg confirming .jpeg images in two-level class subdirectory layout.

3.3 Preprocessing (train.py lines 30-44)

- image_size=(224, 224) -- resized to 224x224 px
- batch_size=8
- label_mode='binary' -- 0/1 integer labels
- shuffle=True for train, shuffle=False for val
- Per-backbone preprocess_input applied inside the model graph (lines 63, 73, 83)

3.4 Data Augmentation (train.py lines 47-53)

- RandomFlip('horizontal') -- horizontal mirror
- RandomRotation(0.1) -- +/-10% rotation (~+/-36°)
- RandomZoom(0.1) -- +/-10% zoom
- Applied via .map() with num_parallel_calls=AUTOTUNE + prefetch(AUTOTUNE)

3.5 Class Imbalance

No class-weighting, oversampling, or undersampling is applied. The Kaggle dataset is significantly imbalanced (~4x more PNEUMONIA than NORMAL in training), which is unaddressed in the code.

4. Model Architecture and Training Setup

4.1 Ensemble Architecture

The ensemble is a late-fusion feature-level concatenation model with a shared input layer. Three backbone branches each apply their own preprocess_input, then pass through the frozen pretrained backbone, a GlobalAveragePooling2D layer, and a Dropout(0.3). The three feature vectors are concatenated (4352-d: DenseNet 1024 + MobileNet 1280 + ResNet 2048), then passed through Dense(256, relu), Dropout(0.4), and Dense(1, sigmoid).

```
Input (224x224x3)
?? preprocess_densenet -> DenseNet121 (frozen) -> GAP -> Dropout(0.3) -> x1 (1024-d)
?? preprocess_mobilenet -> MobileNetV2 (frozen) -> GAP -> Dropout(0.3) -> x2 (1280-d)
```

PneumoScan AI -- Repository Analysis Report

```
?? preprocess_resnet -> ResNet50V2 (frozen) -> GAP -> Dropout(0.3) -> x3 (2048-d)
                      Concatenate([x1, x2, x3]) -> 4352-d
                      Dense(256, relu)
                      Dropout(0.4)
                      Dense(1, sigmoid) -> Prediction
```

4.2 Phase 1 Training -- Feature Extraction (top layers only)

Hyperparameter	Value
Optimizer	Adam
Learning rate	1e-3
Loss	Binary Crossentropy
Metrics	Accuracy, AUC
Epochs (max)	10
Batch size	8
Callback	EarlyStopping(patience=3, restore_best_weights=True)

4.3 Phase 2 Training -- Fine-Tuning (last 20 layers unfrozen per backbone)

Hyperparameter	Value
Optimizer	Adam
Learning rate	1e-5 (10x lower)
Epochs (max)	5
Callback	Same EarlyStopping object reused (see Bug #11)

4.4 Model Saving

- Ensemble saved as models/pneumonia_ensemble_model.h5 (legacy HDF5 format) -- train.py line 140
- OOD model saved as models/ood_isolation_forest.pkl via joblib.dump -- train.py line 164

4.5 OOD Detector

- Feature source: MobileNetV2 branch post-GAP+Dropout output (x2) -- train.py line 144
- Algorithm: IsolationForest(contamination=0.01, random_state=42) -- train.py line 160
- Training data: up to 2,000 chest X-ray features from training set -- train.py lines 148-155
- Interpretation: +1 = inlier (X-ray), -1 = outlier (not X-ray) -- app.py line 69

5. Evaluation Methodology and Metrics

5.1 During Training

Keras logs loss, accuracy, auc and their val_ counterparts after each epoch. EarlyStopping monitors val_loss (not val_auc). AUC is tracked but does not drive early stopping.

5.2 No Formal Evaluation Phase

There is no standalone evaluation script and no test-set evaluation. The project does not compute confusion matrix, precision, recall, F1, sensitivity, or specificity -- standard metrics for medical ML. The available Kaggle test split is entirely unused.

5.3 OOD Evaluation (scratch.py)

scratch.py performs a basic sanity check: iterates all validation images and reports inlier vs. outlier counts. Does not

compute OOD precision/recall against a non-X-ray image set.

5.4 API Test (test_upload.py)

A minimal smoke test sending one known-good X-ray to /predict and printing the JSON response. Not a proper assertion-based test.

6. Inference / Prediction Workflow

6.1 Server Startup (app.py lines 21-48)

- Loads models/pneumonia_ensemble_model.h5 into ensemble_model
- Reconstructs a standalone feature extractor from the MobileNetV2 branch via get_layer('mobilenet_block')
- Loads models/ood_isolation_forest.pkl into ood_model

6.2 Request Processing POST /predict (app.py lines 50-96)

```
POST /predict (multipart/form-data, field: 'file')
?? Content-type check (must start with 'image/')
?? PIL.Image.open().convert('RGB').resize((224,224))
?? img_to_array -> expand_dims -> shape (1,224,224,3)
?? STEP 1: OOD Check
?   features = feature_extractor.predict(image_array)
?   is_inlier = ood_model.predict(features)[0]
?   if -1: return {is_xray: false, ...}
?? STEP 2: Ensemble Prediction
    prediction_prob = ensemble_model.predict(image_array)[0][0]
    result_class = 'Pneumonia' if prob > 0.5 else 'Normal'
    confidence = prob if Pneumonia else (1 - prob)
    return {is_xray: true, prediction: ..., confidence: round(conf*100,2)}
```

6.3 Frontend (static/script.js)

- Drag-and-drop or file browse triggers handleFile() -- image preview via FileReader.readAsDataURL
- Analyze Scan click builds FormData, POSTs to /predict via fetch API
- Three mutually exclusive result states: placeholder, loading, success, invalid
- Confidence bar animates via CSS width transition (1.2s cubic-bezier)

7. Instructions to Run

7.1 Install

```
git clone <repo-url>
cd pneumonia-detection
pip install -r requirements.txt
```

7.2 Dataset Setup

Obtain the Kaggle Chest X-Ray dataset and place it as chest_xray/train/NORMAL/, chest_xray/train/PNEUMONIA/, chest_xray/val/NORMAL/, chest_xray/val/PNEUMONIA/. Or modify DATASET_DIR in train.py line 20.

7.3 Train

```
python train.py
# Outputs: models/pneumonia_ensemble_model.h5 and models/ood_isolation_forest.pkl
```

7.4 Validate OOD (optional)

```
python scratch.py
```

7.5 Start Server

```
python app.py  
# Starts uvicorn on 0.0.0.0:8000
```

7.6 Access Web UI

```
http://localhost:8000
```

7.7 Smoke-test API (optional)

```
python test_upload.py # Requires running server and valid val image path
```

8. Dependencies and Environment

8.1 requirements.txt (7 entries)

Package	Min Version	Role
tensorflow	>=2.10.0	Model building, training, inference
fastapi	>=0.100.0	Async HTTP API framework
uvicorn	>=0.23.0	ASGI server for FastAPI
python-multipart	>=0.0.6	File upload parsing in FastAPI
scikit-learn	>=1.3.0	IsolationForest OOD detector
numpy	>=1.24.0	Array operations
Pillow	>=10.0.0	Image loading and resizing

joblib (used in train.py line 4 and app.py line 7) is NOT listed -- it is a transitive dependency of scikit-learn but this is an implicit assumption. requests (used in test_upload.py) is also absent.

8.2 Python Version

No python_requires specified. TensorFlow >=2.10 supports Python 3.7-3.10. TF 2.10 is the last version with native Windows GPU support; newer versions require WSL2.

8.3 GPU/CUDA Expectations

train.py lines 9-18 probe for GPUs and enable memory growth. Comment on line 8 mentions DirectML -- indicating the author developed on Windows with an AMD/Intel GPU. CPU fallback is graceful (line 18).

8.4 Model Format

Models are saved in deprecated .h5 format (train.py line 140). TF 2.x warns this may be removed; recommended format is SavedModel (directory).

9. Reproducibility and Experiment Tracking

Aspect	State
Random seed (NumPy/TF global)	NOT set -- only IsolationForest has random_state=4
Data shuffle seed	Not set -- image_dataset_from_directory(shuffle=Tr
Experiment tracking	NONE -- no MLflow, W&B, TensorBoard, or logging
TensorBoard callbacks	Absent

PneumoScan AI -- Repository Analysis Report

Training logs	Console stdout only
Model versioning	Overwrite-on-retrain -- only one version kept
Configuration file	NONE -- all hyperparameters hardcoded in train.py
requirements.txt pinning	Uses >= lower bounds only, not pinned

Results are NOT reproducible due to missing global seeds and no version tracking.

10. Code Quality and Maintainability

Strengths

- Clean, readable code -- each file is single-purpose and under 170 lines
- Correct preprocessing -- per-backbone preprocess_input inside the model graph
- Consistent layer naming enables get_layer() in inference correctly
- AUTOTUNE parallelism in dataset pipeline
- Two-phase fine-tuning with learning rate reduction
- Medical disclaimer explicit in UI, README, and JS

Weaknesses

- Hardcoded hyperparameters -- no config file or CLI args
- No logging framework -- only print() statements
- No tests -- test_upload.py is a one-off manual script with no assertions
- No error handling in training -- GPU OOM crash loses all progress (no ModelCheckpoint)
- app.py line 52: dead comment '# ... (rest of the predict function)' is a copy-paste artifact
- Feature extractor code duplicated across app.py, scratch.py, and train.py (DRY violation)
- @app.on_event('startup') is deprecated in FastAPI >=0.93
- No input validation beyond MIME type -- no file size limit
- reload=True in app.py line 103 -- development setting, not for deployment

11. Security, Privacy, Licensing, and Medical ML Caveats

Security

- No authentication or rate limiting on /predict -- open to anyone on the network
- Arbitrary file upload accepted -- malformed/adversarial images could exploit PIL or TF
- No HTTPS -- medical images transmitted unencrypted
- reload=True with uvicorn in production is a security risk

Privacy

- Uploaded images processed in-memory (io.BytesIO) and not persisted -- good for privacy
- No explicit privacy policy or data retention statement

Licensing

- No LICENSE file -- default is All Rights Reserved, restricting redistribution
- Kaggle dataset (CC BY 4.0) is not attributed anywhere in the README
- ImageNet pretrained weights (Keras) are under Apache 2.0

Medical ML Caveats

- Trained on JPEG-compressed X-rays from a single dataset; not validated for DICOM, different scanners, paediatric populations, or different pneumonia aetiologies
- OOD detector trained only on chest X-rays -- may fail to reject similar medical images
- UI claims 'Supports DICOM' (index.html line 100) but PIL cannot open .dcm files -- false claim
- Confidence score is the raw sigmoid output, not a calibrated probability
- No Grad-CAM, SHAP, or uncertainty quantification provided

12. Gaps, Bugs, and Risks Found

#	Severity	Description	File/Line
1	Critical	app.py rebuilds GAP+Dropout layers with random wei	app.py 36-42
2	Critical	joblib used in train.py & app.py but missing from	requirements.txt
3	High	No ModelCheckpoint -- crash during Phase 2 loses a	train.py
4	High	Class imbalance unaddressed (~4x PNEUMONIA vs NORM	train.py
5	High	UI advertises DICOM support; PIL cannot open .dcm	index.html:100
6	High	@app.on_event('startup') deprecated in FastAPI >=0	app.py:21
7	High	.h5 model format deprecated in TF 2.x	train.py:140
8	Medium	No test-set evaluation; Kaggle test split unused	train.py
9	Medium	No global random seeds -- not reproducible	train.py
10	Medium	No file size limit on /predict -- memory risk with	app.py
11	Medium	EarlyStopping object reused across Phase 1+2 -- ma	train.py:137
12	Medium	IsolationForest contamination=0.01 semantically in	train.py:160
13	Low	Dead comment '# ... (rest of the predict function)	app.py:52
14	Low	reload=True in production uvicorn call	app.py:103
15	Low	No LICENSE file	root
16	Low	Kaggle CC BY 4.0 dataset not attributed	README.md
17	Low	README says 'cd pneumonia_app' but dir is pneumoni	README.md:16

13. Prioritised Improvement Roadmap

Priority 1 -- Critical Fixes (Correctness)

- Fix feature extractor in app.py: save the feature extractor as a separate artefact during training (train.py line 144 already builds it) and load it directly -- eliminating the fragile weight-copy approach.
- Add joblib to requirements.txt (e.g. joblib>=1.3.0).
- Fix EarlyStopping reuse: create a fresh EarlyStopping object for Phase 2 fit().

Priority 2 -- High Impact Improvements

- Add ModelCheckpoint to both training phases to prevent total loss on crash.
- Address class imbalance: add class_weight={0: 2.89, 1: 1.0} to model.fit() calls.
- Add a proper test-set evaluation script (evaluate.py) computing accuracy, AUC, precision, recall, F1, sensitivity, specificity, confusion matrix.
- Fix the DICOM claim: remove 'DICOM' from the UI or add pydicom and implement .dcm conversion.

PneumoScan AI -- Repository Analysis Report

- Migrate `@app.on_event('startup')` to a lifespan context manager.
- Switch to SavedModel format: `ensemble_model.save('models/pneumonia_ensemble_model')`.

Priority 3 -- Reproducibility and Observability

- Set global seeds: `tf.random.set_seed(42)` and `np.random.seed(42)` at top of `train.py`.
- Add TensorBoard callback and/or MLflow/W&B; at minimum write training history to JSON/CSV.
- Add a configuration layer: move all hyperparameters to `config.py` or a YAML/JSON config file.

Priority 4 -- Security and Robustness

- Add file size limit: reject uploads above ~20 MB in `/predict` before reading.
- Add authentication/API key for any non-localhost deployment.
- Disable `reload=True`; document using nginx/caddy with TLS for networked deployment.

Priority 5 -- Quality, Documentation, and Legal

- Add a LICENSE file (MIT or Apache 2.0 recommended).
- Add Kaggle CC BY 4.0 dataset attribution to README.
- Fix README typo: `'pneumonia_app'` -> `'pneumonia-detection'`.
- Extract shared feature extractor logic into `utils.py` to eliminate code duplication.
- Add Grad-CAM visualisation to highlight lung regions driving the prediction.
- Add probability calibration (e.g. Platt scaling) for well-calibrated confidence scores.